KFA Public Library was a hit. Membership has grown fast, and the fixed-size, array-only version you built last week is starting to creak: shelves overflow, searching the catalogue is slow, and there is no easy way to stop the same book being "borrowed twice" by mistake. Today you scale it up using the Java Collections Framework, while sharpening the String and Array skills that got you this far.

Continue inside the same KFALibrary project from last week's tutorial (or a fresh copy of it if you'd rather start clean). Keep every class from Sections A–D as you move through them — later sections depend on earlier ones.

General Instructions

- Work individually. You may use your Intellij IDE, the JDK documentation, and your own class notes — **no AI tools**.
- Comment your code briefly wherever the logic isn't obvious; uncommented "magic" logic loses clarity marks.
- Each section builds on the one before it — do not delete earlier classes, only extend them.
- Compile and test each class before moving to the next section.
- Submit one zipped folder named RollNo_KFALibrary.zip containing all .java files and a screenshot folder of each program's console output.
- Upload before the 12 noon of the assigned Tutorial day.

Section A

A1. Data cleaning: sanitizeTitle()

- Write a static method sanitizeTitle(String raw) that: trims leading/trailing whitespace, collapses any run of multiple spaces into a single space, and converts the result to Title Case (first letter of every word capitalised, rest lowercase).
- Solve this using only split(), trim(), substring(), toUpperCase()/toLowerCase() and a loop or StringBuilder — no regex.
- Test it on at least 3 messy inputs, e.g. " the GREAT gatsby " → "The Great Gatsby".

A2. Receipts and the == trap

- Write generateReceiptText(String memberName, LibraryItem item) that builds a multi-line formatted receipt using StringBuilder (member name, item title, due date placeholder, a dashed separator line).
- Separately, write a short demo comparing two ISBN strings with == versus .equals() — construct one with new String(...) so the two clearly disagree — and print both results.
- In 2–3 sentences, explain why the results differ and why .equals() is the correct choice for comparing ISBNs.

Section B

B1. The shelf grid

- Model the physical library as String[5][10] shelves — 5 shelves, 10 slots each — storing ISBNs (use "" for an empty slot).
- Write placeOnShelf(String[][] shelves, String isbn) that places the ISBN in the first empty slot it finds (scanning shelf by shelf, slot by slot) and returns the shelf/slot position, or -1,-1 if the grid is full.
- Write printShelves(String[][] shelves) that prints the grid in a readable row-by-row layout.

B2. Scans and in-place edits

- Write findMostExpensive(Book[] books) that returns the priciest Book using a plain linear scan — no Arrays.sort, no Collections.
- Write reverseInPlace(Book[] books) that reverses the array's element order without creating a second array (swap using a temp variable).
- Demonstrate both methods on a Book[] of at least 5 books and print before/after results.

Section C

The shelf grid works for physical space, but the catalogue itself needs to grow and shrink freely. Migrate it.

C1. Migrate to ArrayList<LibraryItem>

- Replace your fixed-size catalogue array with an ArrayList<LibraryItem> catalogue.
- Implement addItem(LibraryItem item), removeItem(String isbn) (returns true/false depending on whether something was actually removed), and searchByTitle(String keyword) returning an ArrayList<LibraryItem> of all case-insensitive matches.

C2. Sorting with Comparator

- Use Collections.sort() with a Comparator to sort the catalogue by price, ascending.
- Then sort it a second way — alphabetically by title — using a different Comparator (a lambda or a separate Comparator implementation).
- Print the catalogue after each sort so the two orderings are clearly visible.

C3. Undo the last removal

- Maintain a second ArrayList<LibraryItem> named removalHistory.
- Whenever removeItem() successfully removes something, push it onto removalHistory instead of discarding it.
- Implement undoRemove() that pops the most recently removed item off removalHistory and re-adds it to catalogue — and prints a message if there is nothing to undo.

Section D

Two last problems: members occasionally try to "double-borrow" a book by mistake, and searching the ArrayList one item at a time is too slow once the catalogue is large. Fix both with the right collection for the job.

D1. HashSet — stop the double-borrow

- Create a HashSet<String> currentlyBorrowed holding the ISBNs of items that are out on loan.
- Write borrow(String isbn) that adds the ISBN to the set and returns true, or returns false (without changing the set) if that ISBN is already present.
- Write returnBook(String isbn) that removes it from the set. Demonstrate a normal borrow, an attempted double-borrow, and a return, printing the outcome of each.

D2. HashMap — instant ISBN lookup

- Build a HashMap<String, Book> isbnIndex mapping every book's ISBN directly to its Book object.
- Write findByIsbn(String isbn) using the map (get()) instead of looping the ArrayList.
- In a short comment, explain in your own words why this lookup is faster than scanning the ArrayList once the catalogue has thousands of items.

D3. HashMap — most-borrowed report

- Given a String[] borrowLog containing book titles in the order they were borrowed over a semester (with repeats), build a HashMap<String, Integer> borrowFrequency counting how many times each title appears.
- Use getOrDefault() to update counts without a separate containsKey() check.
- Scan the map to find and print the single most-borrowed title and its count.

Submission Checklist

- All .java files compile with zero errors.
- Catalogue fully migrated to ArrayList in Section C — no leftover fixed-size array version.
- Console screenshots included for Sections A, B, C, and D runs.
- Section A2 written explanation (== vs equals) saved as answers.txt or answers.docx inside the same zip.
- Zip named RollNo_KFALibrary_Level2.zip and uploaded before time runs out.